Name: Mrigaj Shaw
Roll No: 2024CSB041

Assignment 5

Problem 1

Problem Statement: Let $S = \{0, 1, 2, \ldots, n-1\}$ be a set of n distinct elements. Initially, each element forms its own separate set, so the collection consists of n disjoint singleton sets. This initial configuration is represented using a parent array P of size n, where $P[i] = i$ for all $0 \leq i \leq n-1$, and a rank array R, where $R[i] = 0$ for all $0 \leq i \leq n-1$.

Design and implement an efficient Disjoint Set (Union–Find) data structure that supports two fundamental operations: `Find(x)`, which returns the representative (root) of the set containing element x, and `Union(x, y)`, which merges the two sets containing elements x and y. To improve efficiency, apply path compression during the `Find` operation so that each visited node directly points to the root, and apply union by rank during the `Union` operation so that the tree with smaller height is attached under the tree with larger height, thereby keeping the tree shallow.

Code

```c
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int *parent;
    int *rank;
    int n;
} DisjointSet;

DisjointSet *createSet(int n) {
    DisjointSet *disjoint = (DisjointSet *)malloc(sizeof(DisjointSet));
    disjoint->n = n;
    disjoint->parent = (int *)malloc(n * sizeof(int));
    disjoint->rank = (int *)malloc(n * sizeof(int));

    for (int i = 0; i < n; i++) {
        disjoint->parent[i] = i;
        disjoint->rank[i] = 0;
    }
    return disjoint;
}

int find(DisjointSet *ds, int i) {
    if (ds->parent[i] != i) {
        ds->parent[i] = find(ds, ds->parent[i]);
    }
    return ds->parent[i];
}

bool unionSets(DisjointSet *ds, int a, int b) {
    int rootA = find(ds, a);
    int rootB = find(ds, b);

    if (rootA != rootB) {
        if (ds->rank[rootA] < ds->rank[rootB]) {
            ds->parent[rootA] = rootB;
        } else if (ds->rank[rootA] > ds->rank[rootB]) {
            ds->parent[rootB] = rootA;
        } else {
            ds->parent[rootB] = rootA;
            ds->rank[rootA]++;
        }
```

```c
        return true;
    }
    return false;
}

bool IsSameSet(DisjointSet *ds, int a, int b) {
    return find(ds, a) == find(ds, b);
}

void freeSet(DisjointSet *ds) {
    free(ds->parent);
    free(ds->rank);
    free(ds);
}

void displaySet(DisjointSet *ds) {
    int n = ds->n;

    printf("Elements: ");
    for (int i = 0; i < n; i++)
        printf("%d ", i);
    printf("\n");

    printf("Parents:  ");
    for (int i = 0; i < n; i++)
        printf("%d ", ds->parent[i]);
    printf("\n");

    printf("Ranks:    ");
    for (int i = 0; i < n; i++)
        printf("%d ", ds->rank[i]);
    printf("\n");
}

int main() {
    int n;
    printf("Enter the number of elements: ");
    scanf("%d", &n);

    DisjointSet *ds = createSet(n);
    printf("Disjoint Set created with %d elements (0 to %d)\n", n, n - 1);

    int choice, a, b;
    do {
        printf("\nDisjoint Set Menu\n");
        printf("1. Display Set\n");
        printf("2. Union\n");
        printf("3. Find (root of element)\n");
        printf("4. Check if Same Set\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
        case 1:
            displaySet(ds);
            break;

        case 2:
            printf("Enter two elements to union: ");
```

```c
            scanf("%d %d", &a, &b);
            if (a < 0 || a >= n || b < 0 || b >= n) {
                printf("Invalid elements. Must be between 0 and %d.\n", n - 1);
                break;
            }
            if (unionSets(ds, a, b))
                printf("Union successful: %d and %d are now in the same set.\n",
                       a, b);
            else
                printf("%d and %d are already in the same set.\n", a, b);
            break;

        case 3:
            printf("Enter element to find root of: ");
            scanf("%d", &a);
            if (a < 0 || a >= n) {
                printf("Invalid element. Must be between 0 and %d.\n", n - 1);
                break;
            }
            printf("Root of %d is: %d\n", a, find(ds, a));
            break;

        case 4:
            printf("Enter two elements to check: ");
            scanf("%d %d", &a, &b);
            if (a < 0 || a >= n || b < 0 || b >= n) {
                printf("Invalid elements. Must be between 0 and %d.\n", n - 1);
                break;
            }
            if (IsSameSet(ds, a, b))
                printf("%d and %d belong to the SAME set.\n", a, b);
            else
                printf("%d and %d belong to DIFFERENT sets.\n", a, b);
            break;

        case 5:
            printf("Freeing memory and exiting...\n");
            freeSet(ds);
            break;

        default:
            printf("Invalid choice. Please try again.\n");
        }
    } while (choice != 5);

    return 0;
}
```

Output

```
Enter the number of elements: 2
Disjoint Set created with 2 elements (0 to 1)

Disjoint Set Menu
1. Display Set
2. Union
3. Find (root of element)
4. Check if Same Set
5. Exit
Enter your choice: 2
Enter two elements to union: 0 1
Union successful: 0 and 1 are now in the same set.

Disjoint Set Menu
1. Display Set
2. Union
3. Find (root of element)
4. Check if Same Set
5. Exit
Enter your choice: 1
Elements: 0 1
Parents:  0 0
Ranks:    1 0
```

Inference

The following conclusions can be drawn from the foundational implementation of the Disjoint Set Union (DSU) data structure:

1. **Data Structure Functionality**:
   - The implementation successfully manages a collection of elements partitioned into non-overlapping (disjoint) subsets, providing a robust mathematical foundation for dynamic connectivity problems.
2. **Operation Mechanics, Path Compression, and Amortized Time**:
   - **Find and Path Compression**: The `find` operation locates the root representative of a set. To drastically optimize this, a technique called *path compression* is employed. During a `find` traversal, every node visited along the path is updated to point directly to the root. This permanently flattens the tree structure, ensuring subsequent accesses are instantaneous.
   - **Union and Amortized Complexity**: The `union(a, b)` function relies entirely on the `find` operation to locate the two roots before linking them. By combining path

compression with union by size/rank, the cost of these operations drops to $O(\alpha(n))$, where $\alpha(n)$ is the Inverse Ackermann function. Because $\alpha(n) \leq 4$ for any computationally feasible input, the **amortized time complexity** is practically $O(1)$. *Amortized time* means that while a single operation might occasionally take longer (e.g., the first deep path compression), the average time per operation over a sequence is mathematically guaranteed to be constant.

3. **Space Allocation**:

   - The structure requires $O(n)$ auxiliary space to store the `parent` array (and optionally the `rank` or `size` arrays) to maintain the state of the n elements.

Problem 2

Problem Statement: In a large school campus, thousands of students dynamically form different friend groups, where new friendships are created daily and teachers frequently need to verify whether two students belong to the same group. Initially, each student forms an independent group. As interactions increase, multiple groups gradually merge, and it becomes essential to efficiently manage these dynamic group connections.

Given n students and m randomly generated friendship operations, design and implement an optimized **Disjoint Set (Union–Find)** data structure using **dynamic arrays**, which supports two fundamental operations:

- `Union(x, y)`: Merges the sets containing students x and y.
- `Find(x)`: Returns the representative (leader) of the set containing student x.

Let the initial parent array be defined as:

$$P[i] = i, \quad \text{for all } 0 \leq i \leq n-1$$

To enhance performance, apply **path compression** during the `Find` operation and **union by rank** during the `Union` operation. The system must efficiently process all m operations, ensuring fast connectivity queries even for large values of n and m.

Code

```c
#include "../DisjointSetUnion/DisjointSetUnion.h"
#include <stdio.h>
#include <string.h>

int main() {
    int n, m;
    printf("Enter number of students and operations: ");
    scanf("%d %d", &n, &m);

    DisjointSet *ds = createSet(n);
    printf("Created set with %d students (0 to %d)\n", n, n - 1);

    for (int i = 0; i < m; i++) {
        char operation[10];
        int a, b;
        scanf("%s", operation);

        if (strcmp(operation, "Union") == 0) {
            scanf("%d %d", &a, &b);
            if (unionSets(ds, a, b))
                printf("Union(%d, %d): merged.\n", a, b);
            else
                printf("Union(%d, %d): already in same group.\n", a, b);
        } else if (strcmp(operation, "Find") == 0) {
            scanf("%d", &a);
            printf("Find(%d): leader is %d\n", a, find(ds, a));
        } else if (strcmp(operation, "Same") == 0) {
            scanf("%d %d", &a, &b);
            if (isSameSet(ds, a, b))
                printf("Same(%d, %d): YES\n", a, b);
            else
                printf("Same(%d, %d): NO\n", a, b);
```

```
        } else {
            printf("Unknown operation.\n");
        }
    }

    freeSet(ds);
    return 0;
}
```

Output

```
Enter number of students and operations: 2 2
Created set with 2 students (0 to 1)
Union 0 1
Union(0, 1): merged.
Find 1
Find(1): leader is 0
```

Inference

Based on the application of DSU to model a social network, the following insights can be highlighted:

1. **Graph Modeling & Algorithmic Mapping**:
   - The problem effectively translates a real-world social network into an undirected graph. Establishing a new friendship maps directly to the `union` operation, seamlessly merging two separate friend groups into a single extended network.
   - Checking if two individuals belong to the same extended friend circle utilizes the `find` operation to verify if they share the identical root representative.
2. **Scalability and Application**:
   - Because the underlying operations operate in amortized constant time (as established in Problem 1), this approach is perfectly suited for dynamic social networks. It provides real-time query responses without the immense computational overhead of performing a full graph traversal (like BFS or DFS) for every single query, keeping operations strictly $O(1)$ on average and space strictly $O(n)$.

Note: `DisjointSetUnion.h` includes the exact same logic and functions implemented and analyzed in Problem 1.

Problem 3

Problem Statement: A city contains N buildings, initially all isolated, which can be mathematically modeled as N disjoint singleton sets, where the parent array is initialized as

$$P[i] = i, \quad \text{for all } 0 \leq i \leq N-1.$$

Over time, M road construction operations are performed, where each operation connects two buildings (u, v), with $0 \leq u, v \leq N-1$. To efficiently manage these dynamic connections, design a **Disjoint Set (Union–Find)** data structure supporting two operations: *Union(u, v)*, which merges the sets containing u and v, and *Find(x)*, which returns the representative (leader) of the set containing x. After each union operation, the system must verify whether the entire city has become fully connected, which mathematically implies that all buildings belong to a single connected component, i.e., there exists a unique representative r such that

$$\text{Find}(i) = r, \quad \text{for all } 0 \leq i \leq N-1.$$

Code

```c
#include "../DisjointSetUnion/DisjointSetUnion.h"
#include <stdio.h>

bool isFullyConnected(DisjointSet *ds) {
    int root = find(ds, 0);
    for (int i = 1; i < ds->n; i++) {
        if (find(ds, i) != root)
            return false;
    }
    return true;
}

int main() {
    int n, m;
    printf("Enter number of buildings and road operations: ");
    scanf("%d %d", &n, &m);

    DisjointSet *ds = createSet(n);
    printf("Created set with %d buildings (0 to %d)\n", n, n - 1);

    for (int i = 0; i < m; i++) {
        int u, v;
        printf("Enter road operation (u v): ");
        scanf("%d %d", &u, &v);

        if (unionSets(ds, u, v))
            printf("Union(%d, %d): connected.\n", u, v);
        else
            printf("Union(%d, %d): already in same component.\n", u, v);

        if (isFullyConnected(ds)) {
            printf("City is now fully connected!\n");
            break;
        } else
            printf("City is not yet fully connected.\n");
    }

    freeSet(ds);
    return 0;
}
```

Output

```
Enter number of buildings and road operations: 2 2
Created set with 2 buildings (0 to 1)
Enter road operation (u v): 0 1
Union(0, 1): connected.
City is now fully connected!
```

Inference

The application of DSU for infrastructural planning (road construction) yields the following technical insights:

1. **Connectivity Verification & Universal Root Matching**:
   - The DSU explicitly verifies the progressive connectivity of the road network by evaluating the collective root state of all buildings.
   - After each road construction (`union` operation), the algorithm isolates the representative root of the first building (building 0) and iterates through all remaining buildings. If every building returns the exact same root, it mathematically proves the entire city has merged into a single connected component.
2. **Infrastructural Suitability**:
   - Evaluating M proposed roads connecting N cities requires minimal overhead. Because the operations utilize path compression (as detailed in Problem 1), both the `union` step and the sequential `find` checks during the connectivity verification happen in amortized constant time. This iterative validation ensures absolute accuracy and demonstrates the foundational logic required for spanning tree validations.

Note: `DisjointSetUnion.h` includes the exact same logic and functions implemented and analyzed in Problem 1.